SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA11

COURSE OUTCOME COVERED

CO1: Apply object-oriented concepts and software development life cycle models to design structured and modular software systems. (BL2, BL3)

Assessment Tool Weightage: 60%

Dr Ramamoorthy

QUESTIONS: 12 Total Marks: 60

Annexure A
CONCEPT MAPPING

Code of Conduct:

I K.Vijayalakshmi (Reg No: 192511189)certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K.Vijayalakshmi

Annexure A
CONCEPT MAPPING

1. Construct a Concept Map

Given:

Requirement Analysis → _____ → Coding → Testing → _____ → Maintenance

Concept Map Diagram:

(Blue boxes = concepts given in the question; Green boxes = missing concepts filled in as the answer)

Answer:

The required concepts are:

- Design
- Deployment

Paraphrased Explanation:

- Design transforms gathered requirements into a detailed blueprint that defines the system architecture, modules, database, and user interface.
- Deployment is the process of delivering the verified software to the production environment so that end users can begin using it.
- Together with the other four stages shown in the diagram, these two missing stages complete the full Software Development Life Cycle (SDLC), which is the standard sequence followed by any software project.

Discussion:

- Following all six phases in the correct order ensures systematic and disciplined software development instead of ad-hoc coding.
- Identifying design problems early, before coding starts, reduces the overall project risk and rework later.
- A proper design phase and a planned deployment phase together improve the quality, reliability, and maintainability of the final software product.

(5 Marks)

2. Construct a Concept Map

Given:

Object-Oriented Principles

→ Abstraction

→ _____

→ Inheritance

→ _____

Concept Map Diagram:

(Blue boxes = concepts given in the question; Green boxes = missing concepts filled in as the answer)

Answer:

The required concepts are:

- Encapsulation
- Polymorphism

Paraphrased Explanation:

- Encapsulation is the principle of wrapping data (attributes) and the methods that operate on that data into a single unit called a class, while hiding the internal details from outside access using access specifiers such as private and public.
- Polymorphism is the principle that allows the same method name or interface to behave differently depending on the object that calls it, for example through method overloading and method overriding.
- Along with Abstraction and Inheritance, which are already given, these four together form the well-known four pillars of Object-Oriented Programming.

Discussion:

- Encapsulation improves data security because internal fields cannot be changed directly from outside the class.
- Polymorphism enhances code reusability, since one interface can serve many different implementations.
- Together, all four pillars support a flexible, efficient, and easy-to-maintain software design.

(5 Marks)

3. Construct a Concept Map

Given:

Class → Blueprint

Object → Instance

Method → _____

Attribute → _____

Concept Map Diagram:

(Blue boxes = concepts given in the question; Green boxes = missing concepts filled in as the answer)

Answer:

The required concepts are:

- Behavior
- State

Paraphrased Explanation:

- A Method represents the Behavior of an object — it defines what actions or operations the object can perform, such as calculate() or display().
- An Attribute represents the State of an object — it stores the current values or properties that describe the object at a given point in time, such as name or balance.
- This mapping shows that a Class acts like a blueprint, and every Object created from it is a real instance that has its own state (attributes) and behavior (methods).

Discussion:

- This concept map helps students clearly understand how object-oriented systems model real-world entities using classes and objects.
- Separating state (attributes) from behavior (methods) makes the object's characteristics and its actions easy to identify.
- It improves the overall organization of the software system by grouping related data and operations together.

Illustrative Example:

A real example: the Car class (blueprint) and myCar (an actual object created from it, with real values).

(5 Marks)

4. A Concept Map Shows

Given:

Software Quality

→ Reliability

→ _____

→ Maintainability

Concept Map Diagram:

(Blue boxes = concepts given in the question; Green boxes = missing concepts filled in as the answer)

Answer:

The missing concept is:

- Usability

Paraphrased Explanation:

- Usability measures how easily and comfortably end users can learn, understand, and interact with the software without confusion.
- Reliability (given) means the software performs its intended functions consistently and correctly without unexpected failures.
- Maintainability (given) means the software is easy to update, fix, or enhance after it has been deployed.

Discussion:

- High usability directly improves user satisfaction, since users can complete their tasks with less effort and fewer errors.
- Good usability, combined with reliability and maintainability, enhances the overall acceptance of the product in the market.
- All three quality attributes together contribute significantly to the long-term success of the software.

(5 Marks)

5. UML Concept Map

Given:

UML

→ Structural Diagrams → _____

→ Behavioral Diagrams → Activity Diagram

→ Interaction Diagrams → _____

Concept Map Diagram:

(Blue boxes = concepts given in the question; Green boxes = missing concepts filled in as the answer)

Answer:

The required concepts are:

- Object Diagram
- Communication Diagram

Paraphrased Explanation:

- An Object Diagram is a type of Structural Diagram that shows a snapshot of the objects in a system at a particular moment, along with their relationships and current attribute values.
- A Communication Diagram (also called a collaboration diagram) is a type of Interaction Diagram that shows how objects exchange messages with one another to accomplish a task.
- Together with the Activity Diagram (given, under Behavioral Diagrams), these three categories cover the main classification of UML diagrams used in object-oriented analysis and design.

Discussion:

- Classifying UML diagrams under Structural, Behavioral, and Interaction categories improves overall system visualization for designers and developers.
- This classification supports object-oriented analysis by separating the static structure of a system from its dynamic behavior.
- Well-organized UML diagrams facilitate clearer communication between developers, designers, and other project stakeholders.

Illustrative Example:

Example of an Object Diagram (a Structural Diagram): two linked object instances with real values.

Example of an Activity Diagram (a Behavioral Diagram): showing a login flow with a decision point.

Example of a Communication Diagram (an Interaction Diagram): numbered messages between two objects.

(5 Marks)

6. Given

Given:

Requirements ✓ → Design ✓ → Coding ✗ → Testing ✗ → Deployment ✗

Concept Map Diagram:

(Green boxes with ✓ = phase completed successfully; Red boxes with ✗ = phase where failure occurred)

Paraphrased Explanation:

- The tick marks against Requirements and Design confirm that these two phases were completed correctly and without any issues.
- The cross mark against Coding indicates that the failure originates in the Coding phase — this is where the actual defect first appears.
- Because Coding failed, the phases that come after it — Testing and Deployment — are also marked with a cross, since a defective program cannot be tested or deployed successfully until the coding errors are fixed.
- The root cause is most likely implementation-level mistakes such as logical errors, incorrect algorithms, or improper use of programming constructs.

Discussion:

- An error introduced in the Coding phase propagates forward and affects every subsequent phase of the SDLC.
- Because the underlying code is faulty, the Testing phase cannot be carried out effectively or produce reliable results.
- Overall software quality decreases, and additional time and cost are needed later to trace and fix the problem.

Solution:

- Review the source code carefully, line by line, to locate the logical or syntax errors.
- Apply consistent coding standards and best practices to reduce the chance of similar mistakes in the future.
- Conduct structured code inspections and peer reviews before the code is handed over to the testing team.

(5 Marks)

7. Construct a Concept Map

Given:

Problem Identification → _____ → Design → Implementation → _____

Concept Map Diagram:

(Blue boxes = concepts given in the question; Green boxes = missing concepts filled in as the answer)

Answer:

The required concepts are:

- Requirement Analysis
- Testing

Paraphrased Explanation:

- Requirement Analysis is the stage that comes right after Problem Identification — here the user's needs and the exact system requirements are studied and documented in detail before any design work begins.
- Testing is the stage that comes right after Implementation — here the completed system is verified and validated to check whether it actually satisfies the requirements that were specified earlier.
- This five-stage flow (Problem Identification, Requirement Analysis, Design, Implementation, Testing) represents the general software development process used to build a system from an initial idea to a working product.

Discussion:

- A proper Requirement Analysis stage improves overall software quality because the design and coding are based on a clear, well-understood problem statement.
- Careful requirement gathering combined with thorough testing at the end reduces the number of development errors and rework.
- Following this structured process from problem identification to testing ensures successful and timely completion of the project.

(5 Marks)

8. Construct a Concept Map

Given:

Object-Oriented Features

→ Class

→ _____

→ Message Passing

→ _____

Concept Map Diagram:

(Blue boxes = concepts given in the question; Green boxes = missing concepts filled in as the answer)

Answer:

The required concepts are:

- Object
- Dynamic Binding

Paraphrased Explanation:

- An Object is a real-world instance created from a Class — the class only defines the structure, while the object holds actual data and can perform actions.
- Dynamic Binding (also called late binding) means that the specific method to be executed is decided at runtime rather than at compile time, which is what makes runtime polymorphism possible.
- This flow — Class, Object, Message Passing, and Dynamic Binding — shows how the basic building blocks of object-oriented systems connect: classes create objects, objects communicate through message passing, and dynamic binding decides how that communication is actually handled at runtime.

Discussion:

- Creating objects from classes and allowing them to communicate through message passing supports a flexible and modular software design.
- Dynamic binding improves code maintainability, since new object types can be added without changing the existing calling code.
- This combination of features enables true runtime polymorphism, one of the most powerful capabilities of object-oriented programming.

(5 Marks)

9. Construct a Concept Map

Given:

Software Development Models

→ Waterfall

→ _____–

→ Spiral

→ _____

Concept Map Diagram:

(Blue boxes = concepts given in the question; Green boxes = missing concepts filled in as the answer)

Answer:

The required concepts are:

- Incremental Model
- Agile Model

Paraphrased Explanation:

- The Incremental Model develops and delivers the software in small stages or builds, with each increment adding more functionality on top of the previous one.
- The Agile Model supports iterative development carried out in short cycles called sprints, with continuous feedback and close involvement from the customer throughout the project.
- Together with the Waterfall model (a strict sequential approach) and the Spiral model (a risk-driven approach), these four represent the major software development life cycle models used in the industry.

Discussion:

- Using models such as Incremental and Agile enhances the project's adaptability to changing requirements.
- Delivering software in smaller stages or sprints reduces overall project risk, since problems are caught early.
- Regular customer involvement, especially in the Agile model, greatly improves customer satisfaction with the final product.

Illustrative Example:

A quick visual comparison of the four models: Waterfall (strict sequence), Incremental (staged builds), Spiral (risk-driven loops), and Agile (repeating sprint cycle).

(5 Marks)

10. Construct a Concept Map

Given:

Class Diagram

→ Class

→ _____

→ Association

→ _____

Concept Map Diagram:

(Blue boxes = concepts given in the question; Green boxes = missing concepts filled in as the answer)

Answer:

The required concepts are:

- Attribute

- Method

Paraphrased Explanation:

- An Attribute defines a property or characteristic of a class, such as name, id, or salary, and represents the data that objects of that class will hold.
- A Method defines an operation or action that a class can perform, such as calculateSalary() or updateRecord(), and represents the behavior of the objects.
- Along with Association (given), which shows the relationship between two classes, these elements together make up the essential components of a UML Class Diagram.

Discussion:

- Showing attributes and methods clearly inside a class diagram represents the internal structure of the system effectively.
- A well-drawn class diagram improves software documentation, making it easier for new developers to understand the system.
- This structured representation strongly supports the overall object-oriented design process.

Illustrative Example:

A real UML class notation box, showing how the class name, attributes, and methods actually appear in a class diagram.

(5 Marks)

11. Construct a Concept Map

Given:

Software Engineering Principles

→ Modularity

→ _____

→ Reusability

→ _____

Concept Map Diagram:

(Blue boxes = concepts given in the question; Green boxes = missing concepts filled in as the answer)

Answer:

The required concepts are:

- Maintainability
- Scalability

Paraphrased Explanation:

- Maintainability is the principle that makes it easier to locate and fix defects, and to update or enhance the software after it has already been deployed.
- Scalability is the principle that allows a system to handle an increasing amount of workload, data, or users efficiently, without needing a complete redesign.

- Together with Modularity (breaking the system into independent modules) and Reusability (given), these four principles guide sound software engineering practice.

Discussion:

- Good maintainability and modularity together improve the overall performance and stability of the software over time.
- Designing for reusability and maintainability reduces the effort needed for future maintenance and bug fixes.
- Building scalability into the system from the start supports the software's ability to evolve over the long term.

(5 Marks)

12. Given

Given:

Requirement Analysis ✓ → System Design ✓ → Coding ✓ → Integration ✗ → Deployment ✗

Concept Map Diagram:

(Green boxes with ✓ = phase completed successfully; Red boxes with ✗ = phase where failure occurred)

Paraphrased Explanation:

- The tick marks against Requirement Analysis, System Design, and Coding confirm that these three phases were carried out correctly, with no issues found in them.
- The cross mark against Integration shows that the failure occurs when the individually coded modules are combined together — the modules do not communicate or work correctly as a combined system.
- Because Integration fails, Deployment is also marked with a cross, since a system whose modules do not integrate properly cannot be safely released to production.
- The likely root cause is mismatched interfaces, incompatible data formats, or incorrect assumptions made independently by different module developers.

Discussion:

- Integration failures typically cause functional inconsistencies between different parts of the system that otherwise worked fine on their own.
- Such failures delay the overall software deployment schedule, since the integration problem must be resolved first.
- Diagnosing integration issues increases debugging effort, as the fault could originate from any of the interacting modules.

Solution:

- Perform dedicated integration testing after individual modules have passed their unit tests.
- Verify that module interfaces, data formats, and communication protocols match the agreed design.
- Resolve any compatibility issues found between modules before attempting deployment again.

(5 Marks)

CONCEPT MAPPING

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrec
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult t interpret in parts